

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники**

**Операционные Системы
Лабораторная работа №1**

Студент: Сидимеков Дмитрий

Группа: Р3322

Преподаватель: Мухаметгалеев Даниил

**САНКТ-ПЕТЕРБУРГ
2025**

Оглавление

Задание 3

Реализация..... 4

Профиллинг и мониторинг..... 4

Вывод..... 21

Задание

Проведите исследование поведения ОС во время исполнения разработанных программ-нагрузчиков по следующему плану:

1. Перед запуском нагрузчика, попробуйте оценить время работы вашей программы или ее результаты (если по варианту вам досталось измерение чего либо) и обоснуйте свои предположения.
2. Запустите программу-нагрузчик и зафиксируйте метрики ее работы с помощью инструментов для мониторинга и профилирования (см. лекции). Сравните полученные результаты с ожидаемыми. Объясните наблюдаемое поведение. Продолжительность каждого запуска должна занимать достаточное для прекращения переходных процессов время, по крайней мере, минуту.
3. Определите количество одновременно запущенных процессов с программой-нагрузчиком, которое эффективно нагружает все ядра процессора в вашей системе. Как распределяются показатели времени USER%, SYS%, WAIT%, а также полное время выполнения нагрузчика, какое количество переключений контекста (вынужденных и невынужденных) происходит при этом? Подумайте над тем, как вы определяете эффективность.
4. Увеличьте количество нагрузчиков вдвое, втрое, вчетверо. Как изменились исследуемые показатели? Почему?
5. Объедините программы-нагрузчики в одну, реализованную при помощи потоков выполнения, чтобы один нагрузчик эффективно нагружал все ядра вашей системы. Как изменились показатели времени для того же объема вычислений? Запустите одну, две, три таких программы. Как изменились исследуемые показатели? Почему?
6. Скомпилируйте программу-нагрузчик с опцией агрессивной оптимизации. Как изменились исследуемые показатели? На сколько сократилось реальное время исполнения программы нагрузчика? Почему?

В процессе защиты вашей работы преподаватель будет просить вас запустить программу-нагрузчик с различными комбинациями параметров и просить объяснить результат.

Реализация

- <https://github.com/sidimekov/os-course/pull/1>

Профилинг и мониторинг

- **cpu-linreg**: алгоритм делает $O(n)$ генераций и суммирований, почти нет системных вызовов, можно ждать высокий USER%, маленький SYS%, почти нет WAIT%, время примерно линейно по $n * repeat$.
- **ema-join-hash**: двойное чтение файла В, построение хэш-таблицы по А, много аллокаций, возможно будет заметный SYS% (fopen, read, write), будет IO-wait если диски медленные. Время зависит от размера входа.
- **io-load**: параметризируемая программа-нагрузчик, которая будет однопоточно нагружать подсистему ввода-вывода (IO).

1. Перед запуском нагрузчика, попробуйте оценить время работы вашей программы или ее результаты (если по варианту вам досталось измерение чего либо) и обоснуйте свои предположения.

В **cpu-linreg** при $n = 10^7$ и repeat порядка сотен время работы составит десятки секунд на одном ядре, так как программа выполняет $O(n * repeat)$ простых операций в памяти и практически не обращается к диску. При увеличении n или repeat время должно расти линейно.

Программа **cpu-ema-hash-join** читает два текстовых файла А и В, строит по файлу А хэш-таблицу и затем несколько раз проходит по файлу В, выполняя поиск и формируя результат. Время работы зависит:

- от суммарного числа строк во входных файлах
- от скорости дисковой подсистемы.

Ожидалось:

- рост времени работы примерно линейно по числу строк
- существенная доля system time и iowait из-за ввода-вывода
- количество добровольных переключений контекста у процесса будет заметно выше, чем у чисто CPU-нагрузки, за счёт блокировок на IO

Программа **io_load** выполняет многократные операции чтения/записи заданного файла с параметрами `block_size`, `block_count`, режимом `read/write`, возможным `O_DIRECT` и последовательным или случайным доступом.

Ожидания:

- при больших объёмах и `direct=on` – почти чистая IO-нагрузка, мало работы в USER, заметный sys и iowait
 - при последовательном доступе – максимальная пропускная способность диска;
 - при случайном доступе – падение скорости из-за большого числа обращений к разным участкам файла;
 - количество добровольных переключений контекста должно быть большим: процесс регулярно блокируется на системных вызовах `pread/pwrite`.
2. Запустите программу-нагрузчик и зафиксируйте метрики ее работы с помощью инструментов для мониторинга и профилирования (см. лекции). Сравните полученные результаты с ожидаемыми. Объясните наблюдаемое поведение. Продолжительность каждого запуска должна занимать достаточное для прекращения переходных процессов время, по крайней мере, минуту.

cpu_lin_reg:

- Команда:

```
vtsh> /usr/bin/time -v ./src/cpu-linreg --n 10000000 --xmin 0 --xmax 100 --ymin 0 --ymax 100 --repeat 250 --quiet on
```

- Вывод:

```
User time (seconds): 57.48
System time (seconds): 0.01
Percent of CPU this job got: 99%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:57.50
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
```

```
Maximum resident set size (kbytes): 1396
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 1
Minor (reclaiming a frame) page faults: 68
Voluntary context switches: 23
Involuntary context switches: 20
Swaps: 0
File system inputs: 48
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Выводы:

- Реальное время (57.5 с) совпадает с **user time**, CPU% = 99% - программа **упирается в вычисления на одном ядре** и практически не простаивает
- System time очень мал (0.01 с), обращений к диску практически нет (Major page faults = 1, File system inputs = 48), как и ожидалось для чисто вычислительного кода.
- Количество переключений контекста очень небольшое (23 добровольных и 20 принудительных), что характерно для этого процесса, который почти не блокируется и лишь периодически вытесняется по таймеру планировщиком.
- Использование памяти минимально (около 1.4 МБ), так как хранятся только несколько сумм и параметры генератора.

В целом измерения подтвердили предварительную оценку: время растёт линейно по $n * repeat$, профиль нагрузки - чистый USER, без заметного IO.

ema_join_hash:

- Команда

```
vtsh> /usr/bin/time -v ./src/cpu-ema-hash-join --a A_1000000.txt --b
B_1000000.txt --out join_out.txt --bucket_count 65536 --repeat 60
```

- Вывод:

```
User time (seconds): 18.78
System time (seconds): 2.57
Percent of CPU this job got: 32%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:05.26
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
```

```
Maximum resident set size (kbytes): 33112
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 1
Minor (reclaiming a frame) page faults: 8145
Voluntary context switches: 205380
Involuntary context switches: 197
Swaps: 0
File system inputs: 56
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Выводы:

- Реальное время составляет 65 с при суммарном user + sys около 21.35 s, при этом CPU% = 32%. Значительную часть времени процесс ждёт завершения **операций ввода-вывода**, а не выполняет вычисления на CPU.
- System time (2.57 s) заметно больше, чем у cpu-linreg, что объясняется активным использованием системных вызовов чтения/записи файлов и работы с файловой системой.
- Очень большое количество **добровольных переключений контекста** (205 380) показывает, что процесс очень часто блокируется на IO и отдаёт процессор другим задачам. Вынужденных переключений (197) существенно меньше т. к. основным источником переключений является именно ожидание диска.
- Max RSS около 32 МБ - ожидаемо больше, чем у cpu-linreg, так как здесь хранится хеш-таблица и буферы для обработки строк.

Метрики хорошо совпадают с ожиданиями для смешанной CPU+IO нагрузки: время выполнения определяется не только объёмом вычислений, но и скоростью диска, а низкий CPU% и большое количество voluntary context switches указывают на серьёзную долю времени в ожидании ввода-вывода.

io_load:

- Команда

```
vtsh> /usr/bin/time -v ./src/io-load --rw=write --block_size=4096 --
block_count=500000 --file=io_test.bin --range=0-0 --direct=off --
type=sequence
```

- Вывод

```
User time (seconds): 0.51
System time (seconds): 6.07
Percent of CPU this job got: 5%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:50.61
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1596
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2
Minor (reclaiming a frame) page faults: 72
Voluntary context switches: 500054
Involuntary context switches: 44
Swaps: 0
File system inputs: 64
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

- Команда

```
vtsh> /usr/bin/time -v ./src/io-load --rw=write --block_size=4096 --
block_count=500000 --file=io_test.bin --range=0-0 --direct=on --
type=sequence
```

- Вывод

```
User time (seconds): 0.40
System time (seconds): 6.67
Percent of CPU this job got: 7%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:38.89
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1572
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2
Minor (reclaiming a frame) page faults: 72
Voluntary context switches: 500035
Involuntary context switches: 50
Swaps: 0
File system inputs: 64
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes)
```

Выводы:

- При реальном времени 110 с суммарное user + sys около 6.6 s, а CPU% = 5%. Это означает, что почти всё время программа проводит в **ожидании диска**: процессор большую часть времени простаивает для данной задачи
- В отличие от cpu-linreg, здесь system time (6.07 s) заметно превышает user time (0.51 s): доминируют системные вызовы и обслуживание файловой системы.
- 500к добровольных переключений контекста – IO нагрузка: процесс почти каждый раз после вызова rwrite блокируется, отдавая CPU другим задачам. Вынужденных переключений мало (44), что логично: планировщик почти не вытесняет процесс, он и так большую часть времени спит
- Использование памяти небольшое (около 1.6 МБ RSS), что соответствует простому буферу фиксированного размера.

Видим поведение чистого IO-нагрузчика: очень маленькое CPU-время, огромная доля времени в ожидании завершения операций записи и огромное количество добровольных переключений контекста.

3. Определите количество одновременно запущенных процессов с программой-нагрузчиком, которое эффективно нагружает все ядра процессора в вашей системе. Как распределяются показатели времени USER%, SYS%, WAIT%, а также полное время выполнения нагрузчика, какое количество переключений контекста (вынужденных и невынужденных) происходит при этом? Подумайте над тем, как вы определяете эффективность.

- Команда

```
vtsh> /usr/bin/time -v bash -c 'for i in 1 2 3 4 5 6 7 8 9 10 11 12;
do ./src/cpu-linreg --n 1000000 --xmin 0 --xmax 100 --ymin 0 --ymax
100 --repeat 250 --quiet on & done; wait'
```

- Вывод

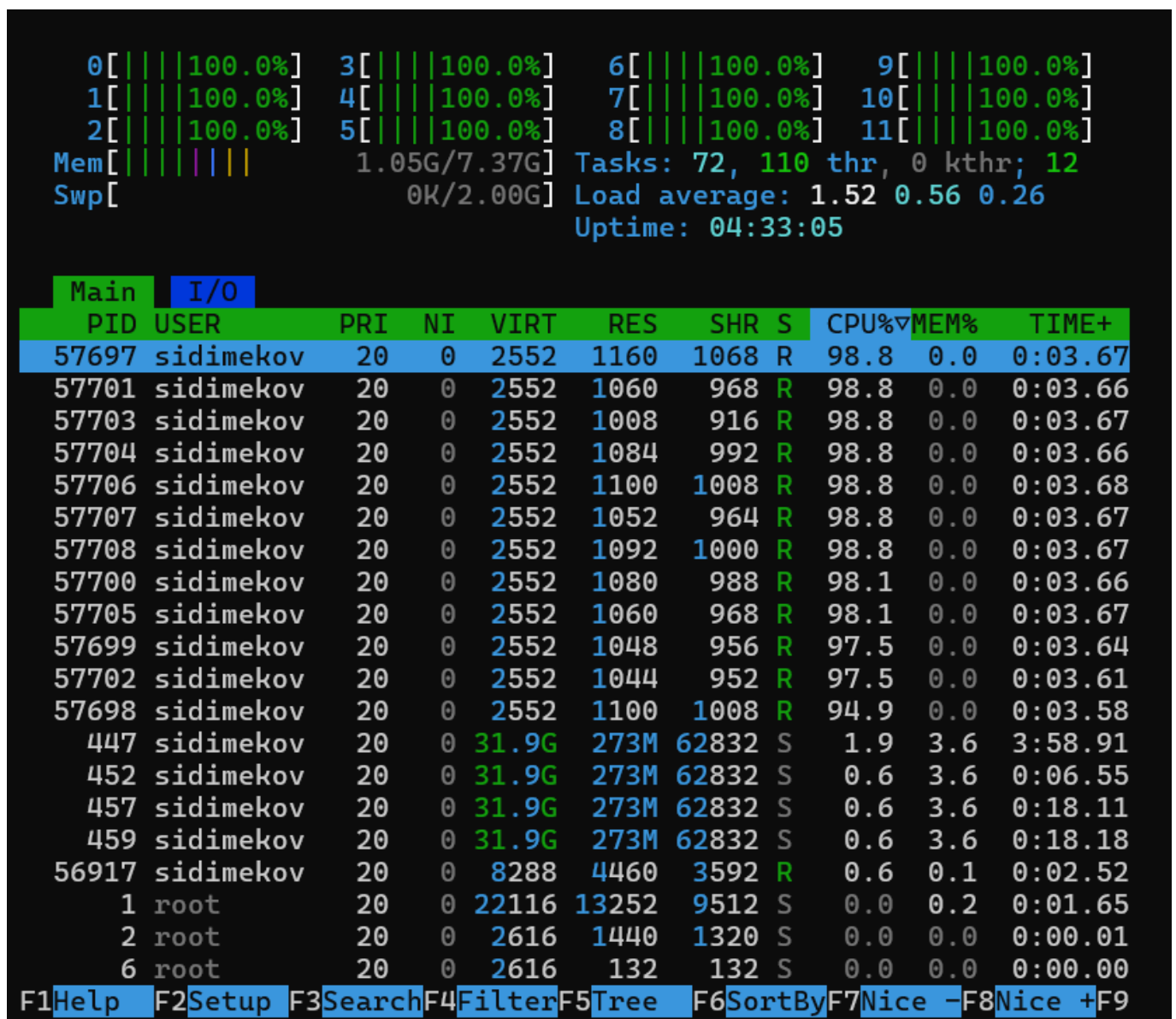
```
User time (seconds): 117.05
System time (seconds): 0.02
Percent of CPU this job got: 1170%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:10.00
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 3856
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 15
```

```

Minor (reclaiming a frame) page faults: 1791
Voluntary context switches: 161
Involuntary context switches: 925
Swaps: 0
File system inputs: 224
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

```

- htop:



Выводы:

- При запуске 12 параллельных процессов cpu-linreg суммарное пользовательское время составило 117.05 с при реальном времени около 10 с, а утилита time показала загрузку CPU 1170%.

- Процессы суммарно использовали около 11.7 ядер-секунд за каждую секунду, то есть практически полностью загрузили все 12 ядер. Системное время (0.02 с) мало по сравнению с пользовательским, что подтверждает чисто вычислительный характер нагрузки.
- Максимальное потребление памяти невелико (3.8 МБ на все процессы), так как каждый процесс использует только небольшой набор буферов и счётчиков.
- Число добровольных переключений контекста остаётся небольшим (161), тогда как вынужденные переключения (925) выше, чем в случае одиночного процесса: планировщик ядра активно делит процессорное время между 12 конкурирующими CPU-bound задачами.
- По htop видно, что все 12 логических процессоров загружены на 98–100% в режиме USER

4. Увеличьте количество нагрузчиков вдвое, втрое, вчетверо. Как изменились исследуемые показатели? Почему?

- Команда с 24 нагрузчиками (при 12 ядрах)

```
vtsh> /usr/bin/time -v bash -c 'for i in $(seq 1 24); do ./src/cpu-
linreg --n 500000 --xmin 0 --xmax 100 --ymin 0 --ymax 100 --repeat 250
--quiet on & done; wait'
```

- Вывод

```
User time (seconds): 117.52
System time (seconds): 0.08
Percent of CPU this job got: 1180%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:09.96
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 3912
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 18
Minor (reclaiming a frame) page faults: 3426
Voluntary context switches: 297
Involuntary context switches: 6756
Swaps: 0
File system inputs: 328
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
```

```
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

- Команда с 36 нагрузчиками (при 12 ядрах)

```
vtsh> /usr/bin/time -v bash -c 'for i in $(seq 1 36); do ./src/cpu-
linreg --n 500000 --xmin 0 --xmax 100 --ymin 0 --ymax 100 --repeat 250
--quiet on & done; wait'
```

- Вывод

```
User time (seconds): 198.47
System time (seconds): 0.33
Percent of CPU this job got: 1186%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:16.75
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 4096
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 16
Minor (reclaiming a frame) page faults: 4998
Voluntary context switches: 434
Involuntary context switches: 20109
Swaps: 0
File system inputs: 48
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

- Команда с 48 нагрузчиками (при 12 ядрах)

```
vtsh> /usr/bin/time -v bash -c 'for i in $(seq 1 48); do ./src/cpu-
linreg --n 500000 --xmin 0 --xmax 100 --ymin 0 --ymax 100 --repeat 250
--quiet on & done; wait'
```

- Вывод

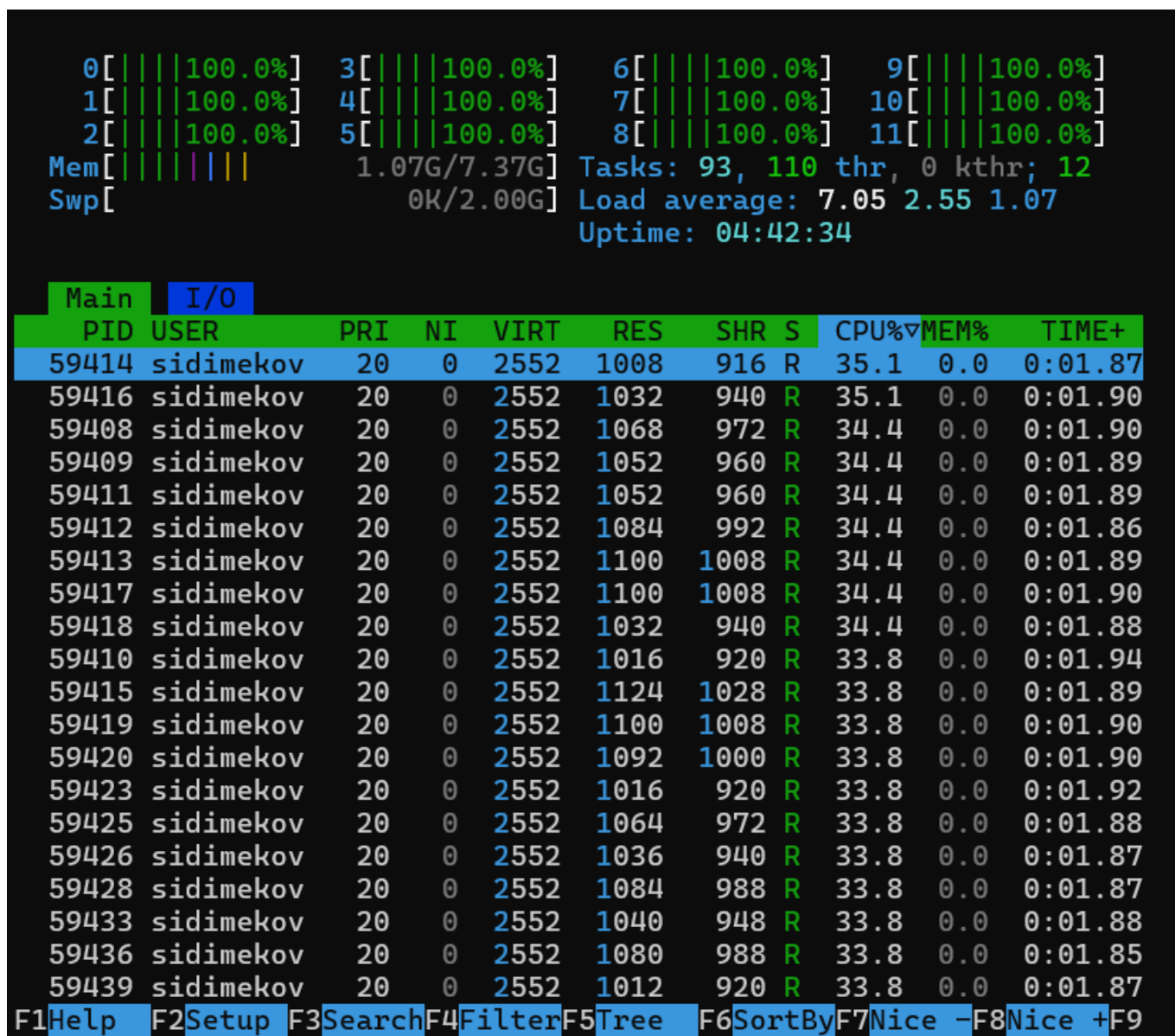
```
User time (seconds): 246.14
System time (seconds): 0.30
Percent of CPU this job got: 1189%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:20.71
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 3924
Average resident set size (kbytes): 0
```

```

Major (requiring I/O) page faults: 18
Minor (reclaiming a frame) page faults: 6511
Voluntary context switches: 537
Involuntary context switches: 25274
Swaps: 0
File system inputs: 48
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

```

- htop:



Выводы:

- Загрузка CPU: 24 процессов - 1176%, 36 процессов - 1186%, 48 процессов - 1189%. Все три значения остаются близкими к пределу - 1200%, что соответствует 12 ядрам.

Добавление новых процессов не увеличивает суммарную загрузку CPU - она ограничена числом ядер. Все новые процессы конкурируют за те же 12 процессоров, поэтому картинка в htop остаётся одинаковой.

- 24 процессов - 9 s, 36 процессов - 16.75 s, 48 процессов - 20.71 s.

При большом количестве процессов система тратит значительную часть времени на переключения и распределение CPU, поэтому эффективность выполнения задач падает, и реальное время перестаёт уменьшаться линейно от снижения n.

- **Voluntary context switches:** 24 процессов - 331, 36 процессов - 434, 48 процессов - 537. Рост небольшой, CPU процессы почти не блокируются.
- **Involuntary context switches:** 24 процессов - 14 362, 36 процессов - 20 109, 48 процессов - 25 274. Здесь наблюдается многопроцессная перегрузка. Чем больше процессов, тем чаще ядро вынуждено вытеснять один процесс ради другого, что приводит к: потере локальности кеша, увеличению задержек выполнения, росту системных накладных расходов.

Переключения растут примерно линейно с количеством процессов - и это является основной причиной падения эффективности при 3х и 4х перегрузке CPU.

5. Объедините программы-нагрузчики в одну, реализованную при помощи потоков выполнения, чтобы один нагрузчик эффективно нагружал все ядра вашей системы. Как изменились показатели времени для того же объема вычислений? Запустите одну, две, три таких программы. Как изменились исследуемые показатели? Почему?

- Команда с 12 потоками (Один нагрузчик)

```
vtsh> /usr/bin/time -v bash -c './src/cpu-linreg-mt --threads=12 --n=1000000 --repeat=250 & wait'
```

- Вывод

- Команда с 12 потоками (Два нагрузчика)

```
vtsh> /usr/bin/time -v bash -c './src/cpu-linreg-mt --threads=12 --n=1000000 --repeat=250 & ./src/cpu-linreg-mt --threads=12 --n=1000000 --repeat=250 & wait'
```

- Вывод

```
User time (seconds): 262.75
System time (seconds): 0.41
Percent of CPU this job got: 1176%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:22.37
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 4008
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 3
Minor (reclaiming a frame) page faults: 606
Voluntary context switches: 53
Involuntary context switches: 14142
Swaps: 0
File system inputs: 64
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

- Htop


```

0[||||100.0%] 3[||||100.0%] 6[||||100.0%] 9[||||100.0%]
1[||||100.0%] 4[||||100.0%] 7[||||100.0%] 10[||||100.0%]
2[||||100.0%] 5[||||100.0%] 8[||||100.0%] 11[||||100.0%]
Mem[|||||] 1.15G/7.37G Tasks: 57, 145 thr, 0 kthr; 12
Swp[|||||] 0K/2.00G Load average: 3.12 1.28 0.48
Uptime: 06:20:39

```

Main	I/O										
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	
78819	sidimekov	20	0	98.6M	1068	976	R	51.0	0.0	0:02.00	
78809	sidimekov	20	0	98.6M	1068	976	R	50.3	0.0	0:02.16	
78811	sidimekov	20	0	98.6M	1068	976	R	50.3	0.0	0:02.02	
78812	sidimekov	20	0	98.6M	1044	948	R	50.3	0.0	0:02.00	
78814	sidimekov	20	0	98.6M	1044	948	R	50.3	0.0	0:02.05	
78815	sidimekov	20	0	98.6M	1068	976	R	50.3	0.0	0:02.14	
78818	sidimekov	20	0	98.6M	1044	948	R	50.3	0.0	0:01.98	
78823	sidimekov	20	0	98.6M	1068	976	R	50.3	0.0	0:02.00	
78827	sidimekov	20	0	98.6M	1068	976	R	50.3	0.0	0:02.14	
78808	sidimekov	20	0	98.6M	1068	976	R	49.7	0.0	0:02.22	
78810	sidimekov	20	0	98.6M	1068	976	R	49.7	0.0	0:02.18	
78816	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:02.15	
78820	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:01.99	
78821	sidimekov	20	0	98.6M	1068	976	R	49.7	0.0	0:01.99	
78822	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:02.00	
78824	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:01.99	
78825	sidimekov	20	0	98.6M	1068	976	R	49.7	0.0	0:02.00	
78828	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:02.09	
78829	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:02.21	
78831	sidimekov	20	0	98.6M	1044	948	R	49.7	0.0	0:02.07	
F1Help	F2Setup	F3Search	F4Filter	F5Tree	F6SortBy	F7Nice	-F8Nice	+F9			

- Команда с 12 потоками (Три нагрузчика)

```

vtsh> /usr/bin/time -v bash -c './src/cpu-linreg-mt --threads=12 --n=1000000 --repeat=250 & ./src/cpu-linreg-mt --threads=12 --n=1000000 --repeat=250 & ./src/cpu-linreg-mt --threads=12 --n=1000000 --repeat=250 & wait'

```

- Вывод

```

User time (seconds): 364.77
System time (seconds): 0.24
Percent of CPU this job got: 1188%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:30.70
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 3972
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 4

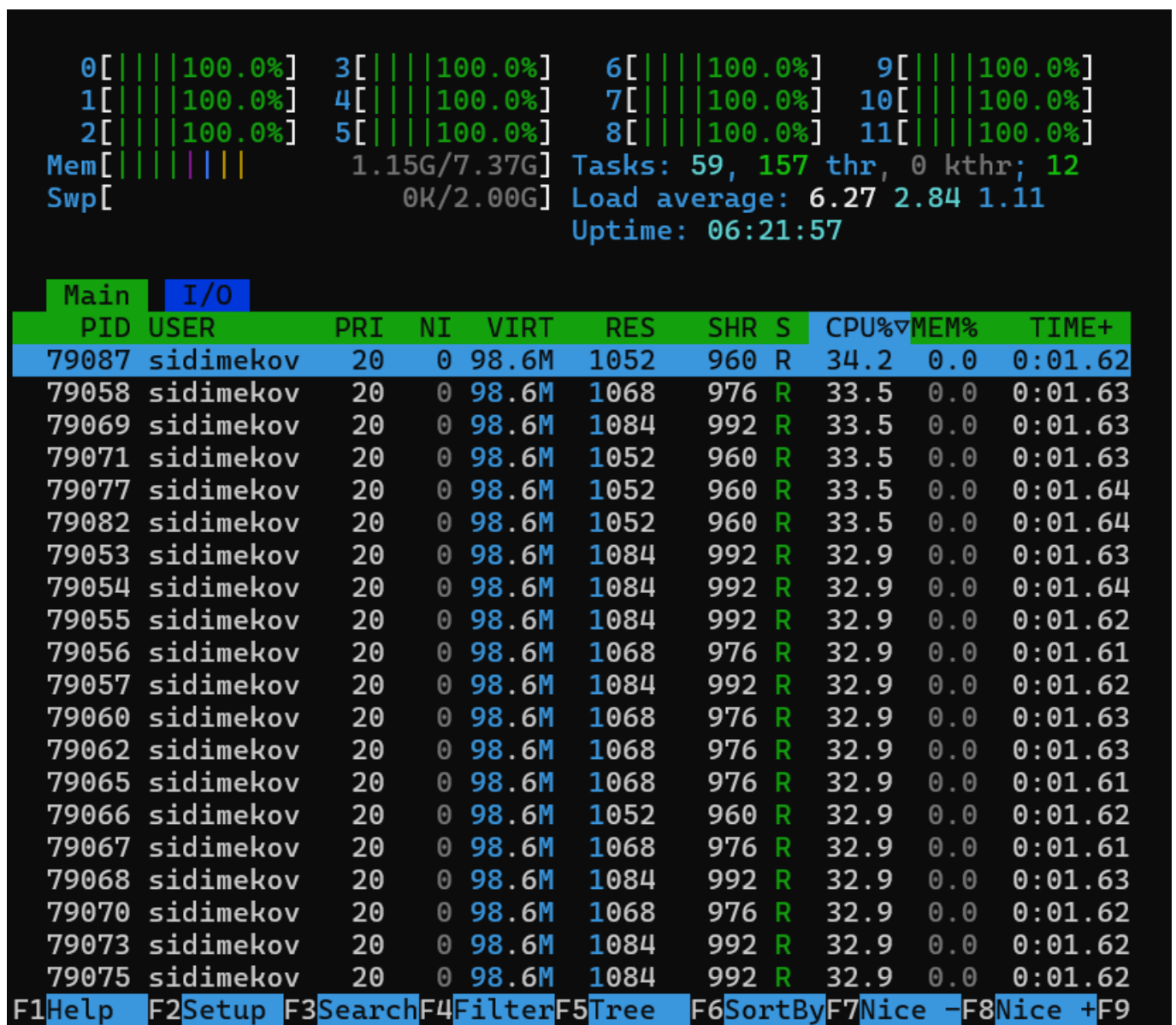
```

```

Minor (reclaiming a frame) page faults: 753
Voluntary context switches: 69
Involuntary context switches: 36058
Swaps: 0
File system inputs: 64
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

```

- Htop



Выводы:

- При запуске `cru-linreg-mt` с числом потоков, равным числу логических ядер (12), по htop видно, что все 12 CPU загружены на 100% в режиме USER, аналогично предыдущему эксперименту с 12 отдельными процессами.

- По `/usr/bin/time -v` реальное время выполнения составило примерно 11 секунд, а суммарное пользовательское время — 122 секунды, что даёт загрузку CPU порядка $(122 / 11) \cdot 100\% = \text{около } 1129\%$. Системное время остаётся минимальным, а количество вынужденных переключений контекста сопоставимо с многопроцессным вариантом. Таким образом, один процесс с несколькими потоками эффективно нагружает все ядра так же, как и набор отдельных процессов.
- При запуске двух и трёх многопоточных нагрузчиков одновременно суммарная загрузка CPU остаётся близкой к 1200% (все 12 ядер постоянно заняты), но реальное время выполнения каждой программы увеличивается: процессорное время распределяется между 24 и 36 потоками
- Суммарное User time по `/usr/bin/time -v` пропорционально числу запущенных программ, количество вынужденных переключений контекста заметно увеличивается. Это связано с тем, что планировщик чаще вытесняет потоки и перераспределяет кванты времени между большим количеством конкурирующих задач.
- В целом поведение похоже на многопроцессный вариант, но многопоточная реализация имеет немного меньшие накладные расходы (общее адресное пространство, меньше переключений контекста между процессами), поэтому при одинаковом объёме работы реальное время выполнения одной многопоточной программы оказывается чуть лучше, чем у набора из такого же числа отдельных процессов.

6. Скомпилируйте программу-нагрузчик с опцией агрессивной оптимизации. Как изменились исследуемые показатели? На сколько сократилось реальное время исполнения программы нагрузчика? Почему?

cpu_lin_reg:

- Команда:

```
vtsh> /usr/bin/time -v ./src/cpu-linreg --n 10000000 --xmin 0 --xmax 100 --ymin 0 --ymax 100 --repeat 250 --quiet on
```

- Вывод:

```
User time (seconds): 14.87
System time (seconds): 0.00
Percent of CPU this job got: 99%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:14.87
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1400
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2
Minor (reclaiming a frame) page faults: 67
Voluntary context switches: 22
Involuntary context switches: 3
Swaps: 0
File system inputs: 64
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

ema_join_hash:

- Команда

```
vtsh> /usr/bin/time -v ./src/cpu-ema-hash-join --a A_1000000.txt --b
B_1000000.txt --out join_out.txt --bucket_count 65536 --repeat 60
```

- Вывод:

```
User time (seconds): 14.69
System time (seconds): 2.55
Percent of CPU this job got: 30%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:56.44
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 33100
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2
Minor (reclaiming a frame) page faults: 8159
Voluntary context switches: 205398
Involuntary context switches: 238
Swaps: 0
File system inputs: 80
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

io_load:

- Команда

```
vtsh> /usr/bin/time -v ./src/io-load --rw=write --block_size=4096 --  
block_count=500000 --file=io_test.bin --range=0-0 --direct=off --  
type=sequence
```

- Вывод

```
User time (seconds): 0.57  
System time (seconds): 5.90  
Percent of CPU this job got: 5%  
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:51.51  
Average shared text size (kbytes): 0  
Average unshared data size (kbytes): 0  
Average stack size (kbytes): 0  
Average total size (kbytes): 0  
Maximum resident set size (kbytes): 1536  
Average resident set size (kbytes): 0  
Major (requiring I/O) page faults: 2  
Minor (reclaiming a frame) page faults: 70  
Voluntary context switches: 500051  
Involuntary context switches: 49  
Swaps: 0  
File system inputs: 72  
File system outputs: 0  
Socket messages sent: 0  
Socket messages received: 0  
Signals delivered: 0  
Page size (bytes): 4096  
Exit status: 0
```

- Агрессивные оптимизации компилятора значительно ускоряют CPU-bound нагрузки, такие как `cru-linreg`: уменьшается количество выполняемых инструкций, лучше используются регистры и векторные инструкции, что приводит к кратному уменьшению User time и соответственно Elapsed time.
- Для смешанных задач (`cru-ema-hash-join`) эффект уже ограничен скоростью дисковой подсистемы, а для чисто IO-bound нагрузки (`io-load`) реальное время практически не меняется, поскольку решающим фактором остаётся время выполнения операций ввода-вывода, а не скорость вычислений на CPU.

Вывод

В ходе лабораторной работы был реализован базовый shell, с реализацией логического оператора `&&` (по варианту) с использованием `clone3` (по варианту). Были реализованы нагрузки `io_load` (нагрузчик подсистемы ввода-вывода),

cpu_linreg (вычислительная), cpu_eta_join_hash (нагрузчик внешней памяти). Проведено исследование поведения ОС под нагрузкой: измерены метрики (USER%, SYS%, WAIT%, контекстные переключения), построено сравнение однопоточной и многопроцессной нагрузки, изучены CPU-bound и IO-bound характеристики.